

ON GENERATING TEST PROBLEMS FOR NONLINEAR PROGRAMMING ALGORITHMS*

RICHARD H. BARTELS[†] AND NEZAM MAHDAVI-AMIRI[‡]

Abstract. We present techniques for the generation of test problems which provide a way of constructing nonlinear programming problems from a wide variety of given functions. These techniques permit one to specify an arbitrary number of points, at each one of which the problem should satisfy some “interesting” conditions (i.e. be optimal or stationary or degenerate), and to determine the characteristics of functions and derivatives at these points (i.e. choose predetermined values for functions, gradients, Hessians and Lagrange multipliers). Moreover, there is a limited capacity to create test problems with special structure.

We give a sample of results obtained by using these techniques to generate test problems for two algorithms to solve problems having nonlinear-least-squares structure with nonlinear constraints.

Key words. optimization, testing, nonlinear constraints, numerical problem generation

1. Introduction. During the last decade a number of methods have been developed for minimizing general objective functions subject to general nonlinear constraints. These methods are characterized by their properties of global convergence, asymptotic superlinear rates of convergence, and formulation in terms of processes that can be efficiently and accurately implemented as numerical algorithms. Examples would include the following references: [5], [6], [15], [16], [25], [26], [29].

No case has to be made for the fact that, in translating these or other methods into mathematical software, a ready supply of test examples can be extremely useful. It is less often recognized that test examples can also be used to probe for oversights in the theory, and can therefore be useful in earlier and more fundamental stages of algorithm development. It was with the intention of exploring ways of making the generation of test examples more flexible that the present work was done. The goal in mind was that it should be easy to craft examples to probe an algorithm's behaviour, producing local “scenario” conditions that test the theoretical assumptions upon which the algorithm is based or that exercise critical sections of the algorithm. We believe that this work represents a step toward that goal.

In § 2 of this paper we review the commonly given necessary and sufficient conditions for a point to be an optimizer of a nonlinear programming problem. In § 3 we propose techniques for perturbing any given function so as to impose upon it a preselected value, gradient, and/or Hessian. These techniques permit the construction of the Hessian in a partitioned form so that its “projections” onto given subspaces can be specified. In § 4 we apply the content of §§ 2 and 3 for the construction of nonlinear programming problems. The material of this section is suggestive rather than comprehensive, and it is oriented toward the crafting of individual problems with any number of “interesting” points: maxima, minima, saddles, degeneracies, etc. In the last two sections we explore whether these techniques are at all amenable to the automatic generation of problems as well as the individualized design of test examples.

* Received by the editors June 16, 1983, and in final revised form April 8, 1985. This research was supported by the Natural Sciences and Engineering Research Council under grant A4076 and A2472 and by a Laidlaw Fellowship administered by the University of Waterloo Computer Science Department. Part of this work was submitted to the Department of Mathematical Sciences, Johns Hopkins University, Baltimore, Maryland, by Nezam Mahdavi-Amiri as a Ph.D. thesis.

[†] Department of Computer Science, University of Waterloo, Waterloo, Ontario, Canada N2L 3G1.

[‡] Department of Computer Science, York University, Downsview, Ontario, Canada M3J 1P3.

In § 5 we give a brief survey of two constrained nonlinear least-squares programs that were written to explore the usefulness of these techniques. In § 6 we give some samples of our experience in automatically constructing problems for these codes.

The idea of devising methods to generate test problems is not new. References [2], [18], [20], [21], [23], [27], [31], [30], [35] provide a sampling of the literature. We believe that this work represents the first one in which multiple points can be specified for the test examples. An example of the specification process, as it might be done by hand, is included for illustration. An automatic generator, using the material to be presented here, has been prepared for the simple case of specifying single points as optimizers, and the techniques had enough flexibility that we were able to impose a special structure on the problems generated.

Test generation methods provide a supplement to collections of optimization problems; e.g. [4], [9], [17], [32]. The following references provide some discussions on software testing, and the reporting of test results, from a broader perspective: [7], [8], [22].

It should be brought out that this paper proposes some tools for generating test examples rather than a single, concrete method. The approach derives from some observations about the chain rule coupled with some conditions of interpolation that can be associated with objective and constraint functions in order to force one or more given points to satisfy predetermined conditions of activity, stationarity, optimality, degeneracy, or conditioning.

2. Optimality conditions. We are interested in solving problems of the form

$$\begin{aligned}
 (2.1) \quad & \underset{x}{\text{minimize}} \phi_0(x) \\
 & \text{such that} \\
 & \phi_j(x) = 0, \quad j \in \mathbf{J}_E, \\
 & \phi_j(x) \geq 0, \quad j \in \mathbf{J}_I,
 \end{aligned}$$

where $\mathbf{J}_E = \{1, \dots, l\}$ is the index set of equality constraints and $\mathbf{J}_I = \{l+1, \dots, l+m\}$ is the index set of inequality constraints. We will denote the combined index set by

$$\mathbf{J} \equiv \mathbf{J}_E \cup \mathbf{J}_I.$$

We denote the index set of the active inequality constraints at any point x by

$$\mathbf{J}_A(x) \equiv \{j \in \mathbf{J}_I: \phi_j(x) = 0\}.$$

Throughout this paper we will assume that all functions are twice continuously differentiable. We first state necessary conditions for a given point to be an optimizer for problem (2.1). In order that these conditions be applicable, constraint qualifications must hold at the point. Appropriate qualifications, and their underlying theory, can be found in [12]. In practice, the constraint qualifications given in this reference (or any other of suitable generality) are difficult to verify computationally. In order to implement algorithms, it is often assumed that the following more strict constraint qualifications hold:

Nondegeneracy assumption. Letting $\bar{x}$ stand for the optimizer, as well as for any point encountered by the algorithm, the vectors

$$(2.2) \quad \nabla \phi_j(\bar{x}), \quad j \in \mathbf{J}_E \cup \mathbf{J}_A(\bar{x})$$

are linearly independent.

This is more than enough to imply satisfaction of all of the constraint qualifications given in [12].

First-order necessary conditions. Let x^* be an optimizer for (2.1), and let an appropriate constraint qualification be satisfied at x^* . Then there exist λ_j^* , $j \in \mathbf{J}$ such that

$$(2.3) \quad \begin{aligned} \nabla \phi_0(x^*) - \sum_{j \in \mathbf{J}_E} \lambda_j^* \nabla \phi_j(x^*) - \sum_{j \in \mathbf{J}_A(x^*)} \lambda_j^* \nabla \phi_j(x^*) &= 0, \\ \lambda_j^* &\geq 0, \quad j \in \mathbf{J}_A(x^*), \\ \phi_j(x^*) &= 0, \quad j \in \mathbf{J}_E, \\ \phi_j(x^*) &\geq 0, \quad j \in \mathbf{J}_I, \\ \lambda_j^* \phi_j(x^*) &= 0, \quad j \in \mathbf{J}_I. \end{aligned}$$

Second-order necessary conditions. Let x^* be an optimizer for (2.1), and let an appropriate constraint qualification be satisfied at x^* . Suppose that there exist λ_j^* satisfying the first-order necessary conditions. Then for all

$$\begin{aligned} z \in \mathbf{R}^n \quad \text{s.t.} \quad z^T \nabla \phi_j(x^*) &= 0, \quad j \in \mathbf{J}_E, \\ z^T \nabla \phi_j(x^*) &= 0, \quad j \in \mathbf{J}_A(x^*) \end{aligned}$$

we have

$$(2.4) \quad z^T \left[\nabla^2 \phi_0(x^*) - \sum_{j \in \mathbf{J}_E} \lambda_j^* \nabla^2 \phi_j(x^*) - \sum_{j \in \mathbf{J}_A(x^*)} \lambda_j^* \nabla^2 \phi_j(x^*) \right] z \geq 0.$$

For x^* to be a strict local minimum of problem (2.1) we need the conditions stated in the following result.

Second-order sufficient conditions. Let (x^*, λ^*) satisfy the first-order necessary conditions and an appropriate constraint qualification. If for every nonzero $z \in \mathbf{R}^n$ such that

$$\begin{aligned} z^T \nabla \phi_j(x^*) &= 0, \quad j \in \mathbf{J}_E, \\ z^T \nabla \phi_j(x^*) &= 0, \quad j \in \mathbf{J}_A(x^*) \quad \text{and} \quad \lambda_j^* > 0, \\ z^T \nabla \phi_j(x^*) &\geq 0, \quad j \in \mathbf{J}_A(x^*) \quad \text{and} \quad \lambda_j^* = 0, \end{aligned}$$

it follows that

$$(2.5) \quad z^T \left[\nabla^2 \phi_0(x^*) - \sum_{j \in \mathbf{J}_E} \lambda_j^* \nabla^2 \phi_j(x^*) - \sum_{j \in \mathbf{J}_A(x^*)} \lambda_j^* \nabla^2 \phi_j(x^*) \right] z > 0,$$

then x^* is a strict local minimum of problem (2.1).

DEFINITION. The function

$$L(x, \lambda) = \phi_0(x) - \sum_{j \in \mathbf{J}} \lambda_j \phi_j(x)$$

is the Lagrangian of the problem.

If we regard ∇ and ∇^2 to be differentiation operators with respect to x alone, and if the columns of the matrix Z form a basis for the orthogonal complement of the space spanned by the vectors $\nabla \phi_j(x^*)$ for $j \in \mathbf{J}_E \cup \mathbf{J}_A(x^*)$, then

(2.3) can be written as $\nabla L(x^*, \lambda^*) = 0$;

(2.4) says that $Z^T \nabla^2 L(x^*, \lambda^*) Z$ will be positive semidefinite at an optimizer;

(2.5) gives us conditions under which $Z^T \nabla^2 L(x^*, \lambda^*) Z$ will be positive definite.

The first-order conditions clearly imply that $\lambda_j^* = 0$ for all $j \in \mathbf{J}_I \setminus \mathbf{J}_A(x^*)$; hence at (x^*, λ^*) the function L can be written in the more explicit form indicated by the first- and second-order conditions.

The idea to be presented for generating test problems is straightforward: the functions $\phi_j(x)$, $j \in \mathbf{J} \cup \{0\}$, will be constructed from given functions $q_j(x)$ to which perturbing functions are added. The perturbing functions are chosen so as to force selected values, gradients, and/or Hessians to be taken on at selected points. It will be seen that this choice is achieved by the introduction of simple conditions of interpolation on the perturbing functions. The values, gradients, and/or Hessians of all of the ϕ_j can, in their turn, all be selected in a coordinated way so as to force the first- and/or second-order conditions to hold at a number of selected points. The choice of the $q_j(x)$ will be left open throughout the paper. Within limits, the choice may serve to provide a desired characteristic or structure (e.g. exponential growth, oscillation, boundedness below, sum of squares format, some degree of discontinuity, etc.).

3. Specifying values, gradients, and Hessians. Take a finite number, $s \geq 1$, of selected points $x^{(1)}, \dots, x^{(s)}$. Let

$$q_j: \mathbf{R}^n \rightarrow \mathbf{R}^1$$

and

$$\delta_{ji}, \gamma_{ji}, \Omega_{ji}: \mathbf{R}^1 \rightarrow \mathbf{R}^1$$

be functions,

$$\alpha_{ji} \in \mathbf{R}^1$$

be scalars,

$$d_{ji}, h_{ji} \in \mathbf{R}^n$$

be vectors, and

$$D_{ji} \in \mathbf{R}^{n \times n}$$

be symmetric matrices.

Consider letting

$$(3.1) \quad \begin{aligned} \phi_j(x) = q_j(x) + \sum_{i=1}^s \{ \alpha_{ji} \delta_{ji} [d_{ji}^T(x - x^{(i)})] + \gamma_{ji} [h_{ji}^T(x - x^{(i)})] \\ + \Omega_{ji} [\tfrac{1}{2}(x - x^{(i)})^T D_{ji} (x - x^{(i)})] \}, \end{aligned}$$

for $j \in \mathbf{J} \cup \{0\}$ and $i = 1, \dots, s$.

The functions ϕ_j will provide us, later on, with the objective and constraint functions for a nonlinear programming problem of the form (2.1). The functions

$$q_j(x), \quad j \in \mathbf{J} \cup \{0\}$$

may be chosen at will to provide some behavioural properties for the ϕ_j .

We will see that the values of the ϕ_j can be determined through the selection of the scalars and vectors

$$\alpha_{ji} \in \mathbf{R}^1 \quad \text{and} \quad d_{ji} \in \mathbf{R}^n,$$

that the gradients can be determined through the selection of the matrices

$$h_{ji} \in \mathbf{R}^n,$$

and that the Hessians can be determined through the selection of the matrices

$$D_{ji} \in \mathbf{R}^{n \times n}.$$

The process of imposing the desired values, gradients, and Hessians on the ϕ_j will require that the functions

$$\Omega_{ji}, \gamma_{ji}, \delta_{ji}$$

satisfy conditions of interpolation to be discussed below.

We begin by establishing that the gradient of ϕ_j is:

$$\begin{aligned} \nabla \phi_j(x) = \nabla q_j(x) + \sum_{k=1}^s \{ \alpha_{ji} \delta'_{ji} [d_{ji}^T(x - x^{(i)})] d_{ji} + \gamma'_{ji} [h_{ji}^T(x - x^{(i)})] h_{ji} \\ + \Omega'_{ji} [\tfrac{1}{2}(x - x^{(i)})^T D_{ji} (x - x^{(i)})] \} \end{aligned} \quad (3.2)$$

and the Hessian of ϕ_j is:

$$\begin{aligned} \nabla^2 \phi_j(x) = \nabla^2 q_j(x) + \sum_{i=1}^s \{ \alpha_{ji} \delta''_{ji} [d_{ji}^T(x - x^{(i)})] d_{ji} d_{ji}^T + \gamma''_{ji} [h_{ji}^T(x - x^{(i)})] h_{ji} h_{ji}^T \\ + \Omega''_{ji} [\tfrac{1}{2}(x - x^{(i)})^T D_{ji} (x - x^{(i)})] D_{ji} (x - x^{(i)}) (x - x^{(i)})^T D_{ji} \\ + \Omega'_{ji} [\tfrac{1}{2}(x - x^{(i)})^T D_{ji} (x - x^{(i)})] D_{ji} \}. \end{aligned} \quad (3.3)$$

Let us introduce the following quantities:

$$\begin{aligned} \theta_{ji}(x) &= d_{ji}^T(x - x^{(i)}), & \theta_{ji}^{(t)} &\equiv \theta_{ji}(x^{(t)}) = d_{ji}^T(x^{(t)} - x^{(i)}), \\ \eta_{ji}(x) &= h_{ji}^T(x - x^{(i)}), & \eta_{ji}^{(t)} &\equiv \eta_{ji}(x^{(t)}) = h_{ji}^T(x^{(t)} - x^{(i)}), \\ \tau_{ji}(x) &= \tfrac{1}{2}(x - x^{(i)})^T D_{ji} (x - x^{(i)}), & \tau_{ji}^{(t)} &\equiv \tau_{ji}(x^{(t)}) = \tfrac{1}{2}(x^{(t)} - x^{(i)})^T D_{ji} (x^{(t)} - x^{(i)}). \end{aligned}$$

Note that $\theta_{ji}^{(i)} = \eta_{ji}^{(i)} = \tau_{ji}^{(i)} = 0$.

In terms of these quantities (3.2) and (3.3) become, respectively,

$$\nabla \phi_j(x) = \nabla q_j(x) + \sum_{i=1}^s \{ \alpha_{ji} \delta'_{ji} [\theta_{ji}(x)] d_{ji} + \gamma'_{ji} [\eta_{ji}(x)] h_{ji} + \Omega'_{ji} [\tau_{ji}(x)] D_{ji} (x - x^{(i)}) \}$$

and

$$\begin{aligned} \nabla^2 \phi_j(x) &= \nabla^2 q_j(x) \\ &+ \sum_{i=1}^s \{ \alpha_{ji} \delta''_{ji} [\theta_{ji}(x)] d_{ji} d_{ji}^T + \gamma''_{ji} [\eta_{ji}(x)] h_{ji} h_{ji}^T \\ &+ \Omega''_{ji} [\tau_{ji}(x)] D_{ji} (x - x^{(i)}) (x - x^{(i)})^T D_{ji} + \Omega'_{ji} [\tau_{ji}(x)] D_{ji} \}. \end{aligned}$$

The vectors d_{ji} and the functions δ_{ji} may be exploited to define $\phi_j(x^{(t)})$, for all j, t . The vectors h_{ji} and the functions γ_{ji} may be used to specify the gradient of $\phi_j(x^{(t)})$ for all j, t . The matrices D_{ji} and the functions Ω_{ji} may be defined so that the Hessian of $\phi_j(x^{(t)})$ is specified for all j and t . To make these observations more concrete, we begin by pointing out how the quantities

$$\phi_j(x^{(t)}), \quad \nabla \phi_j(x^{(t)}), \quad \nabla^2 \phi_j(x^{(t)})$$

depend upon the values of the functions

$$\delta_{ji}, \quad \gamma_{ji}, \quad \Omega_{ji}$$

and their first and second derivatives at the argument values

$$\theta_{ji}^{(t)}, \quad \eta_{ji}^{(t)}, \quad \tau_{ji}^{(t)}.$$

Some critical assumptions must be made about the points $x^{(i)}$, $i = 1, \dots, s$ in order to proceed. For a fixed j the points must interact with the vectors d_{ji} , $i = 1, \dots, s$ so that

$$\theta_{ji}^{(t)} = d_{ji}^T (x^{(t)} - x^{(i)}) \neq 0$$

for all $i, t = 1, \dots, s$ but $i \neq t$. Similarly, for h_{ji} and D_{ji} , $i = 1, \dots, s$, we must have

$$\eta_{ji}^{(t)} = h_{ji}^T (x^{(t)} - x^{(i)}) \neq 0$$

and

$$\tau_{ji}^{(t)} = \frac{1}{2}(x^{(t)} - x^{(i)})^T D_{ji} (x^{(t)} - x^{(i)}) \neq 0$$

for $i, t = 1, \dots, s$ but $i \neq t$. These assumptions frequently hold for small s , even for randomly chosen x 's, d 's, h 's and random nonsingular D 's. Should the assumption fail to hold, it will be necessary to re-select an offending x , or alternatively revise a d , h , or D .

Now, if a fixed index t is selected, and if the functions δ_{ji} , γ_{ji} , and Ω_{ji} have been selected so that the following conditions hold

$$\delta_{ji}[\theta_{ji}^{(t)}] = 0 \quad \text{for all } i \neq t, i = 1, \dots, s,$$

$$\phi_{ji}[\theta_{ji}^{(t)}] = \delta_{ji}[0] = 1,$$

$$\gamma_{ji}[\eta_{ji}^{(t)}] = 0 \quad \text{for all } i = 1, \dots, s \text{ (including } t),$$

and

$$\Omega_{ji}[\tau_{ji}^{(t)}] = 0 \quad \text{for all } i = 1, \dots, s \text{ (including } t),$$

then, from (3.1), $\phi_j(x^{(t)})$ reduces to

$$(3.4) \quad \phi_j(x^{(t)}) = q_j(x^{(t)}) + \alpha_{jt}.$$

The above conditions on the functions δ , γ , and Ω are the conditions of interpolation mentioned previously. It should be clear from (3.1) why these conditions have their particular form and why they lead to (3.4). Since the choice of these functions is up to us, we may construct them so that the given values of 1 and 0 are taken on at the given points θ , η , and τ . Using these observations, we see that the value of $\phi_j(x^{(t)})$ can be specified through the selection of α_{jt} .

A similar discussion involving interpolated values of 1 and 0 derived from (3.2) for δ'_{ji} , γ'_{ji} , Ω'_{ji} will lead to the equation

$$(3.5) \quad \nabla \phi_j(x^{(t)}) = \nabla q_j(x^{(t)}) + h_{jt}$$

from which the choice of h_{jt} will serve to specify $\nabla \phi_j(x^{(t)})$. Finally, a discussion involving interpolated values of 1 and 0 derived from (3.3) for δ''_{ji} , γ''_{ji} , and Ω''_{ji} , will serve to specify $\nabla^2 \phi_j(x^{(t)})$ as

$$(3.6) \quad \nabla^2 \phi_j(x^{(t)}) = \nabla^2 q_j(x^{(t)}) + D_{jt}.$$

It should be remarked that, if any functional value $\phi_j(x^{(t)})$ is not to be set specifically, then the corresponding function δ_{jt} may be set identically to zero. This choice of δ_{jt} would not violate any conditions of interpolation that may arise from other specifications, since those conditions only insist that the values, first, or second derivatives of δ_{jt} are to be zero at other points. Likewise, γ_{jt} or Ω_{jt} may be set identically to zero when the corresponding gradient or Hessian is not demanded.

If all of the conditions of interpolation stated or alluded to above are collected together, we find that the following must be imposed upon δ_{ji} :

$$\begin{aligned}
 (3.7) \quad & \delta_{ji}[0] = 1, \\
 & \delta_{ji}[\theta_{ji}^{(t)}] = 0 \quad \text{for all } i \neq t, \text{ and for all } j \text{ such that } \phi_j \text{ is specified at } x^{(t)}, \\
 & \delta'_{ji}[\theta_{ji}^{(t)}] = 0 \quad \text{for all } i, \text{ and for all } j \text{ such that } \nabla \phi_j \text{ is specified at } x^{(t)}, \\
 & \delta''_{ji}[\theta_{ji}^{(t)}] = 0 \quad \text{for all } i, \text{ and for all } j \text{ such that } \nabla^2 \phi_j \text{ is specified at } x^{(t)}.
 \end{aligned}$$

All other values and derivatives of δ_{ji} are immaterial.

Likewise, for γ_{ji} :

$$\begin{aligned}
 (3.8) \quad & \gamma'_{ji}[0] = 1, \\
 & \gamma_{ji}[\eta_{ji}^{(t)}] = 0 \quad \text{for all } i, \text{ and for all } j \text{ such that } \phi_j \text{ is specified at } x^{(t)}, \\
 & \gamma'_{ji}[\eta_{ji}^{(t)}] = 0 \quad \text{for all } i \neq t, \text{ and for all } j \text{ such that } \nabla \phi_j \text{ is specified at } x^{(t)}, \\
 & \gamma''_{ji}[\eta_{ji}^{(t)}] = 0 \quad \text{for all } i, \text{ and for all } j \text{ such that } \nabla^2 \phi_j \text{ is specified at } x^{(t)}.
 \end{aligned}$$

And finally for Ω_{ji} :

$$\begin{aligned}
 (3.9) \quad & \Omega'_{ji}[0] = 1, \\
 & \Omega_{ji}[\tau_{ji}^{(t)}] = 0 \quad \text{for all } i, \text{ and for all } j \text{ such that } \phi_j \text{ is specified at } x^{(t)}, \\
 & \Omega'_{ji}[\tau_{ji}^{(t)}] = 0 \quad \text{for all } i \neq t, \text{ and for all } j \text{ such that } \nabla \phi_j \text{ or } \nabla^2 \phi_j \\
 & \quad \text{is specified at } x^{(t)}, \\
 & \Omega''_{ji}[\tau_{ji}^{(t)}] = 0 \quad \text{for all } i \neq t, \text{ and for all } j \text{ such that } \nabla^2 \phi_j \text{ is specified at } x^{(t)}.
 \end{aligned}$$

Programmatically:

- (1) Choose $x^{(1)}, \dots, x^{(s)}$.
- (2) For each $t = 1, \dots, s$
 - (2.a) To specify ϕ_j at $x^{(t)}$, select a value for $\phi_j(x^{(t)})$, choose d_{jt} at will, and set

$$\alpha_{jt} = \phi_j(x^{(t)}) - q_j(x^{(t)}).$$
 - (2.b) To specify $\nabla \phi_j$ at $x^{(t)}$, select a vector for $\nabla \phi_j(x^{(t)})$ and set

$$h_{jt} = \nabla \phi_j(x^{(t)}) - \nabla q_j(x^{(t)}).$$
 - (2.c) To specify $\nabla^2 \phi_j$ at $x^{(t)}$, select a symmetric matrix for $\nabla^2 \phi_j(x^{(t)})$ and set

$$D_{jt} = \nabla^2 \phi_j(x^{(t)}) - \nabla^2 q_j(x^{(t)}).$$
- (3) For all i, j , and $t \neq i$
 - (3.a) Determine the quantities

$$\begin{aligned} \theta_{ji}^{(t)} &= d_{ji}^T (x^{(t)} - x^{(i)}), \\ \eta_{ji}^{(t)} &= h_{ji}^T (x^{(t)} - x^{(i)}), \\ \tau_{ji}^{(t)} &= \frac{1}{2} (x^{(t)} - x^{(i)})^T D_{ji} (x^{(t)} - x^{(i)}). \end{aligned}$$
 - (3.b) Check that each of these quantities is nonzero. In the (infrequent) event that a zero occurs for any $i, j, t \neq i$, either $x^{(i)}$ or $x^{(t)}$ must be discarded, or d_{ji} , h_{ji} , or D_{ji} must be revised. Return to step (1).

- (4) Collect together from (3.7), (3.8), and (3.9) all of the conditions of interpolation that are required by the specifications made in step (2) above, and construct any functions δ_{ji} , γ_{ji} , Ω_{ji} satisfying those conditions.

This outline provides only the background for constructing test examples. The arbitrary determination of values, gradients, and Hessians is not enough. We proceed in the next section to suggest ways of determining $\phi_j(x^{(i)})$, $\nabla \phi_j(x^{(i)})$, and $\nabla^2 \phi_j(x^{(i)})$ to take account of the material in § 2.

As a closing remark, it should be stressed that the functions δ , γ , and Ω can be drawn from any class of functions for which it makes sense to set up and solve the Hermite interpolation problems listed above. Reference [1] gives algorithms for doing this in case we choose to use polynomials. Other function classes are possible, however, such as splines, trigonometric functions, etc. If the functions q are also chosen from the same class, the structure of that class, if any, may colour the problem as a whole. We illustrate this in a trivial way at the end of the paper by arranging to generate some least-squares problems.

4. Constructing test examples. The optimization problem (2.1) takes on the following form when ϕ_j is structured as in (3.1):

$$\begin{aligned} \underset{x}{\text{minimize}} \quad \phi_0(x) \equiv & q_0(x) + \sum_{i=1}^s \{ \alpha_{0i} \delta_{0i} [d_{0i}^T(x - x^{(i)})] + \gamma_{0i} [h_{0i}^T(x - x^{(i)})] \\ & + \Omega_{0i} [\tfrac{1}{2}(x - x^{(i)})^T D_{0i} (x - x^{(i)})] \}, \end{aligned}$$

such that

$$\begin{aligned} \phi_j(x) \equiv & q_j(x) + \sum_{i=1}^s \{ \alpha_{ji} \delta_{ji} [d_{ji}^T(x - x^{(i)})] + \gamma_{ji} [h_{ji}^T(x - x^{(i)})] \\ & + \Omega_{ji} [\tfrac{1}{2}(x - x^{(i)})^T D_{ji} (x - x^{(i)})] \} = 0, \quad j \in \mathbf{J}_E, \\ \phi_j(x) \equiv & q_j(x) + \sum_{i=1}^s \{ \alpha_{ji} \delta_{ji} [d_{ji}^T(x - x^{(i)})] + \gamma_{ji} [h_{ji}^T(x - x^{(i)})] \\ & + \Omega_{ji} [\tfrac{1}{2}(x - x^{(i)})^T D_{ji} (x - x^{(i)})] \} \geq 0, \quad j \in \mathbf{J}. \end{aligned}$$

Let $(x^{(t)}, \lambda^{(t)})$ be a chosen Lagrange pair, $t \in \{1, \dots, s\}$. The point $x^{(t)}$ will be “interesting” if this pair interacts in some substantial way with the first- and second-order conditions given by (2.3), (2.4), and (2.5). Specifying that $x^{(t)}, \lambda^{(t)}$ will satisfy all of the equalities and inequalities of these conditions is, of course, the way of specifying that $x^{(t)}$ shall be one of the optimizers of (2.1), but there are other possibilities. Lagrange pairs that violate some subset of the equalities or inequalities (or that satisfy more than their fair share of these equalities and inequalities or that do not satisfy enough of the inequalities in a strict sense) are also of interest in providing stationary points, saddle points, problems with degeneracies, problems without feasibilities, etc.

In this presentation it is neither possible nor desirable to consider all of the conditions that one might wish to impose upon an (x, λ) pair. We will restrict ourselves to a general discussion about how to cause $x^{(t)}$ to be an optimizer. Then we will close with a concrete example in which we specify two points for a problem in two variables, choosing one of the points to be optimal and the other to be stationary. Hopefully, this will be sufficient to suggest other possibilities.

4.1. Specifying optimizers. For $x^{(t)}$ to be one of the optimizers of problem (2.1), we are interested, primarily, in making sure that $x^{(t)}$ is feasible, that $\lambda^{(t)}$ is “in kilter”,

that $\nabla L(x^{(t)}, \lambda^{(t)}) = 0$, and that $Z^T \nabla^2 L(x^{(t)}, \lambda^{(t)}) Z$ is positive definite for an appropriate matrix Z . This will make it our primary concern to specify nonnegative values for $\phi_j, j \in \mathbf{J}_b$, zero values for $\phi_j, j \in \mathbf{J}_E \cup \mathbf{J}_{A(x^{(t)})}$, the zero vector for ∇L , and a specified matrix for $\nabla^2 L$. These last two specifications will be made on L indirectly by first setting all of the $\phi_j, j \in \mathbf{J}$, and then setting $\nabla \phi_0$ and $\nabla^2 \phi_0$ appropriately.

- (1) Choose functions q_0 and q_j for $j \in \mathbf{J} \cup \{0\}$. Choose $\mathbf{J}_E$ and $\mathbf{J}_I$.
- (2) For each $t = 1, \dots, s$
 - (2.a) Choose $\mathbf{J}_{A^{(t)}} \subseteq \mathbf{J}_I$ to be the index set denoting the active inequality constraints at $x^{(t)}$.
 - (2.b) Select $x^{(t)}$ and $\lambda^{(t)}$. The values $\lambda_j^{(t)}, j \in \mathbf{J}$ are to be consistent with $\mathbf{J}_E, \mathbf{J}_b$, and $\mathbf{J}_{A^{(t)}}$ in the sense that $\lambda_j^{(t)} = 0, j \in \mathbf{J}_I \setminus \mathbf{J}_{A^{(t)}}$ and $\lambda_j^{(t)} \geq 0, j \in \mathbf{J}_{A^{(t)}}$.
 - (2.c) Set

$$\alpha_{jt} = -q_j(x^{(t)}), \quad j \in \mathbf{J}_E \cup \mathbf{J}_{A(x^{(t)})}$$

and

$$\alpha_{jt} > -q_j(x^{(t)}), \quad j \in \mathbf{J}_I \setminus \mathbf{J}_{A(x^{(t)})}.$$

- (2.d) Choose d_{ji} so that $\theta_{ji}^{(t)} = d_{ji}^T(x^{(t)} - x^{(i)}) \neq 0$ for all j and all $i \neq t$.
- (3) Functions δ_{ji} must be constructed for $j \in \mathbf{J}$ and $i = 1, \dots, s$ so that they satisfy a simplified form of (3.7), namely:

$$\delta_{ji}[0] = 1 \quad \text{for all } i,$$

$$\delta_{ji}[\theta_{ji}^{(t)}] = 0 \quad \text{for all } i \neq t.$$

- (4) Define

$$\phi_j(x) \equiv q_j(x) + \sum_{i=1}^s \alpha_{ji} \delta_{ji}[d_{ji}^T(x - x^{(i)})].$$

- (5) A gradient must be imposed upon ϕ_0 so that $\nabla L(x^{(t)}, \lambda^{(t)}) = 0$ for each $t = 1, \dots, s$. Set

$$h_{0t} = -\nabla q_0(x^{(t)}) + \sum_{j \in \mathbf{J}} \lambda_j^{(t)} \nabla \phi_j(x^{(t)}).$$

- (6) A Hessian must be imposed upon ϕ_0 so that $\nabla^2 L(x^{(t)}, \lambda^{(t)})$ is predetermined for each $t = 1, \dots, s$.

- (6.a) Choose $\nabla^2 L(x^{(t)}, \lambda^{(t)})$ as the desired Hessian of L .

- (6.b) Set

$$D_{0t} = \nabla^2 L(x^{(t)}, \lambda^{(t)}) - \nabla^2 q_0(x^{(t)}) + \sum_{j \in \mathbf{J}} \lambda_j^{(t)} \nabla^2 \phi_j(x^{(t)}).$$

- (7) Determine whether

$$\eta_{0i}^{(t)} = h_{0i}^T(x^{(t)} - x^{(i)}) \neq 0,$$

$$\tau_{0i}^{(t)} = \frac{1}{2}(x^{(t)} - x^{(i)})^T D_{0t}(x^{(t)} - x^{(i)}) \neq 0$$

for all $i \neq t$. If any of these quantities are zero, return to (2) to select new x 's.

(8) Functions γ_{0i}, Ω_{0i} must be constructed for $i = 1, \dots, s$ so that

$$\begin{aligned}\gamma'_{0i}[0] &= 1, \\ \gamma'_{0i}[\eta_{0i}^{(t)}] &= 0 \quad \text{for all } i \neq t, \\ \gamma''_{0i}[\eta_{0i}^{(t)}] &= 0 \quad \text{for all } i \text{ (including } t), \\ \Omega'_{0i}[\tau_{0i}^{(t)}] &= 0 \quad \text{for all } i \neq t, \\ \Omega'_{0i}[0] &= 1, \\ \Omega''_{0i}[\tau_{0i}^{(t)}] &= 0 \quad \text{for all } i \neq t.\end{aligned}$$

(9) Define

$$\phi_0(x) = q_0(x) + \sum_{i=1}^s \{ \gamma_{0i}[h_{0i}^T(x - x^{(i)})] + \Omega_{0i}[\frac{1}{2}(x - x^{(i)})^T D_{0i}(x - x^{(i)})] \}.$$

Remarks. (a) Vectors can be selected arbitrarily to define $\nabla \phi_j(x^{(t)})$ for $j \in \mathbf{J}$. By setting

$$h_{ji} = \nabla \phi_j(x^{(t)}) - \nabla q_j(x^{(t)})$$

and by checking that $h_{ji}^T(x^{(t)} - x^{(i)}) \neq 0$ for all j and $i \neq t$, functions γ_{ji} can be constructed according to the conditions of interpolation given in (3.8) in order to impose the desired gradients on the equality and active-inequality constraint functions. Degenerate problems can be attempted by choosing gradient vectors which are linearly dependent.

The definition of each ϕ_j in step (4) above for which $\nabla \phi_j$ is to be specified will have to be expanded to include appropriate terms of the form

$$\gamma_{ji}[h_{ji}^T(x - x^{(i)})].$$

Similarly, symmetric matrices can be selected arbitrarily to define $\nabla^2 \phi_j(x^{(t)})$ for $j \in \mathbf{J}$. The definition of each ϕ_j in step (4) above for which $\nabla^2 \phi_j$ is to be specified will have to be expanded to include appropriate terms of the form

$$\Omega_{ji}[\frac{1}{2}(x - x^{(i)})^T D_{ji}(x - x^{(i)})].$$

(b) The $\lambda_j^{(t)}$ may be chosen arbitrarily, consistent only with the inequalities of (2.3). The selection of $\lambda_j^{(t)} = 0$ for one or more $j \in \mathbf{J}_A(x^{(t)})$ often provides difficulty for algorithms.

(c) If the matrix $\nabla^2 L$ is chosen to be positive definite, then the inequalities of (2.4) and (2.5) will be satisfied automatically.

(d) Indefinite or ill-conditioned test problems can be obtained by letting

$$h_{ji} = -\nabla q_j(x^{(t)}) + w_j^{(t)}, \quad j \in \mathbf{J}_E \cup \mathbf{J}_{A^{(t)}},$$

where the vectors $w_j^{(t)}$ form an orthonormal set. This will result in:

$$\nabla \phi_j(x^{(t)}) = w_j^{(t)}, \quad j \in \mathbf{J}_E \cup \mathbf{J}_{A^{(t)}}.$$

Let $W^{(t)}$ stand for the matrix whose columns are given by the $w_j^{(t)}$; let the vectors $z_l^{(t)}$ constitute a basis for the orthogonal complement of the space spanned by the $w_j^{(t)}$, and let $Z^{(t)}$ represent the matrix whose columns are given by the $z_l^{(t)}$. Consequently

$$\begin{bmatrix} W^{(t)T} \\ Z^{(t)T} \end{bmatrix} \begin{bmatrix} W^{(t)} & Z^{(t)} \end{bmatrix} = \begin{bmatrix} I & 0 \\ 0 & I \end{bmatrix}.$$

Then let

$$\nabla^2 L(x^{(t)}, \lambda^{(t)}) = \begin{bmatrix} W^{(t)} & Z^{(t)} \end{bmatrix} \begin{bmatrix} \sigma V_1^{(t)} & 0 \\ 0 & \tau V_2^{(t)} \end{bmatrix} \begin{bmatrix} W^{(t)T} \\ Z^{(t)T} \end{bmatrix} \begin{matrix} \}^{l+|J_{A^{(t)}}|} \\ \}_{n-l-|J_{A^{(t)}}|} \end{matrix}$$

with positive-definite matrices $V_i^{(t)}$, $i = 1, 2$. The matrix $\nabla^2 L$ will be indefinite if and only if one of σ or τ is nonpositive. For any nonzero vector $y \in \mathbf{R}^n$ we may write

$$y = y_1 + y_2,$$

where y_1 is in the space spanned by $w_j^{(t)}$ and y_2 is orthogonal to this space. The equality

$$y^T \nabla \phi_j(x^{(t)}) = 0, \quad j \in J_E \cup J_{A^{(t)}},$$

is equivalent to $y_1 = 0$ since $\nabla \phi_j(x^{(t)}) = w_j^{(t)}$. This leads to the equation

$$y^T \nabla^2 L(x^{(t)}, \lambda^{(t)}) y = \tau y_2^T V_2^{(t)} y_2,$$

which guarantees the satisfaction of the second-order conditions for all values of σ , provided that $\tau > 0$. For ill-conditioned examples, $V_1^{(t)}$ and $V_2^{(t)}$ can be chosen accordingly. This scheme of generating $\nabla^2 L$ in a partitioned format in order to obtain ill-conditioned projected Hessians is necessary in the light of the results contained in [34].

4.2. Something specific. We close this section with a concrete example which uses the material under discussion in a more ambitious way.

Let

$$q_0(x) = q_0(x_1, x_2) = -e^{-(x_1^2 - x_2^2)},$$

$$q_1(x) = q_1(x_1, x_2) = 5 - x_1^3 x_2^3,$$

$$q_2(x) = q_2(x_1, x_2) = 2 - x_1^4 - x_2^4.$$

We will use these functions to construct ϕ_0 , ϕ_1 , and ϕ_2 for the problem

$$\underset{x}{\text{minimize}} \phi_0(x)$$

subject to

$$\phi_1(x) \geq 0,$$

$$\phi_2(x) \geq 0$$

so that at the point

$$x^{(1)} = \begin{bmatrix} x_1^{(1)} \\ x_2^{(1)} \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

we will have

$$\phi_1(x^{(1)}) = 0,$$

$$\phi_2(x^{(1)}) > 0,$$

$$\nabla \phi_1(x^{(1)}) = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

$$\lambda_1^{(1)} = 0,$$

$$\lambda_2^{(1)} = 1,$$

$$\nabla L(x^{(1)}, \lambda^{(1)}) = 0$$

and

$$\nabla^2 L(x^{(1)}, \lambda^{(1)}) = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

The ϕ 's are further to be constructed so that at the point

$$x^{(2)} = \begin{bmatrix} x_1^{(2)} \\ x_2^{(2)} \end{bmatrix} = \begin{bmatrix} -1 \\ -1 \end{bmatrix}$$

we will have

$$\begin{aligned} \phi_1(x^{(2)}) &> 0, \\ \phi_2(x^{(2)}) &= 0, \\ \lambda_1^{(2)} &= 1, \\ \lambda_2^{(2)} &= -1 \end{aligned}$$

and

$$\nabla L(x^{(2)}, \lambda^{(2)}) = 0.$$

The above (subject to *post facto* verification) should cause $x^{(1)}$ to be a degenerate optimizer and $x^{(2)}$ to be a stationary point.

Since we wish to set the value of ϕ_1 at both $x^{(1)}$ and $x^{(2)}$, and its gradient at $x^{(1)}$, we need to set

$$\phi_1(x) \equiv q_1(x) + \alpha_{11} \delta_{11} [d_{11}^T(x - x^{(1)})] + \alpha_{12} \delta_{12} [d_{12}^T(x - x^{(2)})] + \gamma_{11} [h_{11}^T(x - x^{(1)})].$$

Since only the value of ϕ_2 is to be set at both points, we need

$$\phi_2(x) \equiv q_2(x) + \alpha_{21} \delta_{21} [d_{21}^T(x - x^{(1)})] + \alpha_{22} \delta_{22} [d_{22}^T(x - x^{(2)})].$$

Finally, the gradient of the Lagrangian is to be set to zero at both points, by setting $\nabla \phi_0$ appropriately, and the Hessian of the Lagrangian is to be set at $x^{(1)}$ by setting $\nabla^2 \phi_0$. This means that

$$\phi_0(x) \equiv q_0(x) + \gamma_{01} [h_{01}^T(x - x^{(1)})] + \gamma_{02} [h_{02}^T(x - x^{(2)})] + \Omega_{01} [\frac{1}{2}(x - x^{(1)})^T D_{01}(x - x^{(1)})].$$

It is convenient to begin by recording the values, gradients, and Hessians which we might need from q_0 , q_1 , and q_2 :

For $x^{(1)}$:

Function	Value	Gradient	Hessian
q_0	-1	$\begin{bmatrix} -2 \\ 2 \end{bmatrix}$	$\begin{bmatrix} -2 & 4 \\ 4 & -6 \end{bmatrix}$
q_1	4	$\begin{bmatrix} -3 \\ -3 \end{bmatrix}$	$\begin{bmatrix} -6 & -9 \\ -9 & -6 \end{bmatrix}$
q_2	0	$\begin{bmatrix} -4 \\ -4 \end{bmatrix}$	$\begin{bmatrix} -12 & 0 \\ 0 & -12 \end{bmatrix}$

For $x^{(2)}$:

Function	Value	Gradient
q_0	-1	$\begin{bmatrix} -2 \\ 2 \end{bmatrix}$
q_1	4	$\begin{bmatrix} 3 \\ 3 \end{bmatrix}$
q_2	0	$\begin{bmatrix} 4 \\ 4 \end{bmatrix}$

We begin by setting the values of ϕ_1 and ϕ_2 . The value $\phi_1(x^{(1)}) = 0$ may be set by letting

$$\alpha_{11} = -q_1(x^{(1)}) = -4.$$

The value $\phi_2(x^{(2)}) = 0$ may be set by letting

$$\alpha_{22} = -q_2(x^{(2)}) = 0.$$

To set $\phi_2(x^{(1)}) > 0$, we let

$$\alpha_{21} > -q_2(x^{(1)}) = 0,$$

specifically:

$$\alpha_{21} = 1.$$

To set $\phi_1(x^{(2)}) > 0$, we let

$$\alpha_{12} > -q_1(x^{(2)}) = -4,$$

specifically:

$$\alpha_{12} = 0.$$

Choose d_{11} , d_{12} , d_{21} , and d_{22} "arbitrarily":

$$d_{11} = d_{21} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

and

$$d_{12} = d_{22} = \begin{bmatrix} -1 \\ -1 \end{bmatrix}.$$

This will mean that

$$d_{ji}^T(x^{(t)} - x^{(i)}) = -4$$

for $j = 1, 2$, $i = 1, 2$, $t = 1, 2$, and $i \neq t$. This (particularly economical) choice of d 's will mean that the functions δ must satisfy only the following conditions of interpolation:

for δ_{11} :

$$\delta_{11}(0) = 1,$$

$$\delta_{11}(-4) = 0,$$

$$\delta'_{11}(0) = 0;$$

for δ_{12} :

$$\begin{aligned}\delta_{12}(0) &= 1, \\ \delta_{12}(-4) &= 0, \\ \delta'_{12}(-4) &= 0;\end{aligned}$$

for δ_{21} :

$$\begin{aligned}\delta_{21}(0) &= 1, \\ \delta_{21}(-4) &= 0;\end{aligned}$$

and δ_{22} :

$$\begin{aligned}\delta_{22}(0) &= 1, \\ \delta_{22}(-4) &= 0.\end{aligned}$$

These conditions will be satisfied if

$$\begin{aligned}\delta_{11}(t) &= 1 - \frac{t^2}{16}, \\ \delta_{12}(t) &= 1 + \frac{t}{2} + \frac{t^2}{16}\end{aligned}$$

and

$$\delta_{21}(t) = \delta_{22}(t) = 1 + \frac{t}{4}.$$

The final stage in the construction of ϕ_1 comes from the requirement that

$$\nabla \phi_1(x^{(1)}) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

This will mean that

$$h_{11} = \nabla \phi_1(x^{(1)}) - \nabla q_1(x^{(1)}) = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - \begin{bmatrix} -3 \\ -3 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix}.$$

In order to proceed, we must verify that $h_{11}^T(x^{(2)} - x^{(1)})$ is nonzero:

$$h_{11}^T(x^{(2)} - x^{(1)}) = [3 \quad 3] \begin{bmatrix} -2 \\ -2 \end{bmatrix} = -12.$$

It follows that the conditions of interpolation on γ_{11} will be

$$\begin{aligned}\gamma_{11}(0) &= 0, \\ \gamma_{11}(-12) &= 0, \\ \gamma'_{11}(0) &= 1\end{aligned}$$

which will be satisfied for

$$\gamma_{11}(t) = t + \frac{t^2}{12}.$$

Combining all of the above results yields:

$$\phi_1(x) = \phi_1(x_1, x_2) = -(x_1^3 x_2^3 - x_2^2 - 2x_1 x_2 + x_2 - x_1^2 + x_1 + 1)$$

and

$$\phi_2(x) = \phi_2(x_1, x_2) = -\frac{4x_2^4 - x_2 + 4x_1^4 - x_1 - 10}{4}.$$

We now tabulate the gradients and Hessians for ϕ_1 and ϕ_2 that we might need.

For $x^{(1)}$:

Function	Gradient	Hessian
ϕ_1	$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} -4 & -7 \\ -7 & -4 \end{bmatrix}$
ϕ_2	$\begin{bmatrix} -\frac{15}{4} \\ -\frac{15}{4} \end{bmatrix}$	$\begin{bmatrix} -12 & 0 \\ 0 & -12 \end{bmatrix}$

For $x^{(2)}$:

Function	Gradient
ϕ_1	$\begin{bmatrix} -2 \\ -2 \end{bmatrix}$
ϕ_2	$\begin{bmatrix} \frac{17}{4} \\ \frac{17}{4} \end{bmatrix}$

To impose the conditions

$$\begin{aligned}\nabla L(x^{(1)}, \lambda^{(1)}) &= 0, \\ \nabla L(x^{(2)}, \lambda^{(2)}) &= 0,\end{aligned}$$

and

$$\nabla^2 L(x^{(1)}, \lambda^{(1)}) = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix},$$

we must set

$$h_{01} = \begin{bmatrix} -\frac{23}{4} \\ -\frac{7}{4} \end{bmatrix}, \quad h_{02} = \begin{bmatrix} -\frac{17}{4} \\ -\frac{33}{4} \end{bmatrix}$$

and

$$D_{01} = \begin{bmatrix} -8 & -3 \\ -3 & -4 \end{bmatrix}.$$

Observe that

$$\begin{aligned}h_{01}^T(x^{(2)} - x^{(1)}) &= \begin{bmatrix} -\frac{23}{4} & -\frac{7}{4} \end{bmatrix} \begin{bmatrix} -2 \\ -2 \end{bmatrix} = 15 \neq 0, \\ h_{02}^T(x^{(1)} - x^{(2)}) &= \begin{bmatrix} -\frac{17}{4} & -\frac{33}{4} \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} = -25 \neq 0,\end{aligned}$$

and

$$\frac{1}{2}(x^{(2)} - x^{(1)})D_{01}(x^{(2)} - x^{(1)}) = \frac{1}{2} \begin{bmatrix} -2 & -2 \end{bmatrix} \begin{bmatrix} -8 & -3 \\ -3 & -4 \end{bmatrix} \begin{bmatrix} -2 \\ -2 \end{bmatrix} = -36 \neq 0.$$

Therefore, to specify ϕ_0 we must construct γ_{01} , γ_{02} , and Ω_{01} to satisfy:

$$\begin{aligned} \gamma'_{01}(0) &= 1, & \gamma'_{02}(0) &= 1, \\ \gamma'_{01}(15) &= 0, & \gamma'_{02}(-25) &= 0, \\ \gamma''_{01}(0) &= 0, & \gamma''_{02}(-25) &= 0, \\ \Omega'_{01}(0) &= 1, \\ \Omega'_{01}(-36) &= 0. \end{aligned}$$

These conditions will be achieved with

$$\begin{aligned} \Omega_{01}(t) &= t + \frac{t^2}{72}, \\ \gamma_{02}(t) &= t + \frac{t^2}{25} + \frac{t^3}{1,875}, \\ \gamma_{01}(t) &= t + \frac{t^3}{675}. \end{aligned}$$

Consequently,

$$\begin{aligned} \phi_0(x) &= \phi(x_1, x_2) \\ &= -e^{x_2^2 - x_1^2} + \frac{x_2^4}{18} + \frac{x_1 x_2^3}{6} - \frac{367,429 x_2^3}{540,000} \\ &\quad + \frac{25 x_1^2 x_2^2}{72} - \frac{284,221 x_1 x_2^2}{180,000} + \frac{1,583 x_2^2}{3,600} + \frac{x_1^3 x_2}{3} \\ &\quad - \frac{100,543 x_1^2 x_2}{60,000} + \frac{1117 x_1 x_2}{1,800} + \frac{15 x_2}{8} + \frac{2 x_1^4}{9} \\ &\quad - \frac{530,021 x_1^3}{540,000} - \frac{7,417 x_1^2}{3,600} + \frac{23 x_1}{8} - \frac{199}{24}. \end{aligned}$$

The exercise above has a value if the behaviour of an algorithm is to be explored under specific conditions, particularly in the early stages of development: How might it perform in the neighbourhood of a degenerate optimum? What difficulties arise with zero λ 's? We feel that the techniques outlined above have their main value in the flexibility with which they provide such "scenarios" for use as "scalpels" in dissecting the local behaviour of algorithms on a microscopic level. However, less elaborate test examples which are produced in a more automatic fashion are of interest, too, to test the implementation of an algorithm at a later stage of development or to use for comparing optimization systems. In view of this, we felt called upon to take a preliminary look at the feasibility of using these techniques in some automatic manner.

Since the techniques present a vast number of possibilities, any automatic generation scheme will have to be made with some restrictions. We began with the simplest subset of possibilities, that of generating problems with a single prescribed point chosen

to be an optimizer. Within this context we have experimented with problems having a special structure (nonlinear least squares) and “difficult” optima (indefinite Lagrangian Hessians with prescribed positive-definite projections).

The authors’ work in progress is aimed at extending our capabilities to the automatic generation of problems with several specified points.

5. Algorithms. In order to have algorithms to motivate the automatic generation of some problems, we have prepared two experimental codes to solve the constrained nonlinear least-squares problem; i.e. problem (2.1) for the special objective function

$$\underset{x}{\text{minimize}} \phi_0(x) = \frac{1}{2}F(x)^T F(x),$$

where

$$F(x) = [f_1(x), \dots, f_p(x)]^T.$$

It is worth noting that this structure in the objective function provides the following structure to the gradient and the Hessian:

$$\nabla \phi_0(x) = J(x)^T F(x),$$

where $J(x)$ is the Jacobian of $F(x)$ (the matrix whose i th row is $\nabla f_i(x)^T$), and

$$\nabla^2 \phi_0(x) = J(x)^T J(x) + \sum_{i=1}^p f_i(x) \nabla^2 f_i(x).$$

We took two of the currently popular methods for nonlinear programming, the successive quadratic programming technique with the watchdog modification due to Han, Powell et al. [15], [16], [26], and the exact penalty technique due to Coleman and Conn [5], [6], and we made some modifications to these methods to take account of the special structure of the objective function. Our modifications were along the lines of those suggested by Dennis et al. in [11], extended to account for constraints. Our reasons for not experimenting with (2.1) directly were twofold: (1) by treating a problem for which there is not widely available software, we were forced to write programs from scratch, which meant that we could code both methods in a reasonably uniform manner—which we hoped would suppress artifacts arising from different coding styles and put both methods on a nearly equal footing, giving us a model situation for algorithm comparison; (2) by making changes to both methods to account for the special nature of the problem, the blame for any poor showing of one method over another could be laid to us for botching the job—we were interested only in the mechanisms of test generation and of its utility to researchers and programmers, not in making enemies by passing judgement on published methods; (3) we wished to explore the capacity of these techniques to impose structure on the problems generated—the least-squares structure was a simple enough one that it was evident to us how it could be achieved.

We will give a brief summary on both optimization methods and indicate the modifications that we made. The references above are to be consulted for further details about unmodified aspects of these methods, references [5], [6], [26] being the most pertinent.

For both methods a point x^0 and an $n \times n$ positive-definite matrix B^0 are needed to begin. The starting point is user-provided and the initial choice of B^0 , dependent upon the technique being used, is some approximation to second derivative information. From then on, a sequence of points $\{x^k\}$ and a sequence of positive-definite matrices $\{B^k\}$ ($k = 1, 2, 3, \dots$) is generated iteratively.

5.1. The successive quadratic programming method with watchdog. At the beginning of the k th iteration, both x^k and B^k are known. A search direction d^k is obtained by solving the quadratic program:

$$\underset{d}{\text{minimize}} \quad d^T \nabla \phi_0(x^k) + \frac{1}{2} d^T B^k d$$

subject to

$$\begin{aligned} \phi_j(x^k) + d^T \nabla \phi_j(x^k) &= 0, & j = 1, \dots, l, \\ \phi_j(x^k) + d^T \nabla \phi_j(x^k) &\geq 0, & j = l+1, \dots, l+m. \end{aligned}$$

The solution of this quadratic program also yields Lagrange multipliers λ^{k+1} . The search direction d^k is taken with a step size α^k to yield a new point $x^{k+1} = x^k + \alpha^k d^k$, and B^{k+1} is defined by

$$B^{k+1} = J(x^{k+1})^T J(x^{k+1}) + S^{k+1}.$$

We have chosen S^{k+1} as a quasi-Newton approximation to:

$$\sum_{i=1}^p f_i(x^{k+1}) \nabla^2 f_i(x^{k+1}) - \sum_{j=1}^{l+m} \lambda_j^{k+1} \nabla^2 \phi_j(x^{k+1}).$$

The choice was made as follows:

In the first iteration set $S^0 \leftarrow 0_{n \times n}$.

For all other iterations find $\nabla \phi_0(x^{k+1}) = J(x^{k+1})^T F(x^{k+1})$ and set

$$\begin{aligned} y^{k+1} &\leftarrow [J(x^{k+1}) - J(x^k)]^T F(x^{k+1}) - \sum_{j=1}^{l+m} \lambda_j^{k+1} [\nabla \phi_j(x^{k+1}) - \nabla \phi_j(x^k)]; \\ s^{k+1} &\leftarrow x^{k+1} - x^k. \end{aligned}$$

Use y^{k+1} and s^{k+1} in the BFGS formula (switching to DFP if using BFGS would result in a division by zero—see, e.g. [10], for a reference to these) to update S^k to S^{k+1} .

In solving the above quadratic programming problem, the following techniques presented in [13] were used:

At each step in finding the optimal value of d a working set of constraints is determined, the gradients of which, $\phi_j(x^k)$, are taken to form the columns of a matrix A .

The columns of the matrix Z are determined to form an orthonormal basis for the null space of A^T .

A modification to the current d is found by solving a system of equations based upon the matrix $Z^T B^k Z$.

The equation-solving process involves using the Cholesky decomposition of

$$(5.1.1) \quad Z^T B^k Z = LDL^T + E,$$

where E is a diagonal matrix of small norm chosen to make $Z^T B^k Z$ positive definite. (E is a zero matrix when $Z^T B^k Z$ is already positive definite.)

After d^k is determined from the quadratic model, the step size α^k is chosen with the aid of two merit functions. The primary of these is

$$\begin{aligned}\psi_k(\alpha) = \Psi(x^k + \alpha d^k, \mu^k) = \phi_0(x) + \sum_{j \in J_E} \mu_j^k |\phi_j(x^k + \alpha d^k)| \\ + \sum_{j \in J_I} \mu_j^k |\min(0, \phi_j(x^k + \alpha d^k))|,\end{aligned}$$

where the parameters μ_j^k for each $j \in J$ were updated according to:

On the first iteration:

$$\mu_j^k = 2|\lambda_{\lambda_j}^{k+1}|.$$

On other iterations:

$$\begin{aligned}\text{If } \mu_j^{k-1} < 1.5|\lambda_j^{k+1}|, \\ \text{then } \mu_j^k = 2|\lambda_j^{k+1}|, \\ \text{else } \mu_j^k = \mu_j^{k-1}.\end{aligned}$$

The secondary merit function used is merely an approximation to the Lagrangian function:

$$L_k(x^k + \alpha d^k) = \phi_0(x^k + \alpha d^k) - \sum_{j \in J} \lambda_j^{k+1} \phi_j(x^k + \alpha d^k).$$

Line searches, as needed, were performed on $\psi_k(\alpha)$ using the line search of [24] with termination set by $\eta = 0.9$. This is a very tight termination requirement that should ensure satisfaction of the line search condition given in [26], namely

$$(5.1.2) \quad \Psi(x^{k+1}, \mu^k) \leq \Psi(x^k, \mu^k) - \theta[\Psi(x^k, \mu^k) - \Psi_k(x^{k+1}, \mu^k)],$$

where θ is a constant chosen from the interval $(0, \frac{1}{2})$. To ensure that the choice $\alpha = 1$ is ultimately always made, we test whether taking the point $x^k + d^k$ as x^{k+1} will ensure that either one of the inequalities (5.1.2) or

$$L_k(x^k + d^k) < L_k(x^k)$$

is satisfied. If so, we accept $x^k + d^k$ as x^{k+1} without carrying out a line search on $\psi_k(\alpha)$.

All other details are identical to those laid out in [26].

The area in which it might be worth probing for a weakness of our implementation of this method is in its use of estimates of Lagrange multipliers at all stages of descent. At points far from any optimizer or stationary point these multipliers may not be valid for use in setting the penalty parameters μ . We expected to use the flexibility of our techniques to try setting local conditions in which the quadratic model would yield large multipliers. This proved unnecessary, as will be seen in the final section. Problems which we generated to explore weaknesses in the second optimization method, to be covered next, also proved to cause a difficulty for the successive quadratic programming method as well.

5.2. The exact penalty method. A single merit function is used by this method:

$$\Psi_\varepsilon(x) = \mu \phi_0(x) - \sum_{j \in J_{V_1}^\varepsilon(x)} \phi_j(x) + \sum_{j \in J_{V_2}^\varepsilon(x)} \text{sgn}(\phi_j(x)) \phi_j(x),$$

where for any x and $\varepsilon > 0$ we define

$J_{V_1}^\varepsilon(x) = \{j: \phi_j(x) < -\varepsilon \text{ and } j = l+1, \dots, l+m\}$ the set of ε -violated inequality constraints,

$J_{V_2}^\varepsilon(x) = \{j: |\phi_j(x)| > \varepsilon \text{ and } j = 1, \dots, l\}$ the set of ε -violated equality constraints.

Implicitly associated with these index sets there is an approximate Lagrangian, which we will denote by

$$L_\varepsilon(x, \lambda) = \Psi_\varepsilon(x) - \sum_{j \in \mathbf{J}_A^\varepsilon(x)} \lambda_j \phi_j(x),$$

where $\mathbf{J}_A^\varepsilon(x)$ complements the index sets $\mathbf{J}_{V_1}^\varepsilon(x)$ and $\mathbf{J}_{V_2}^\varepsilon(x)$ as follows:

$\mathbf{J}_{A_1}^\varepsilon(x) = \{j: |\phi_j(x)| \leq \varepsilon \text{ and } j = l+1, \dots, l+m\}$ the set of ε -active inequality constraints,

$\mathbf{J}_{A_2}^\varepsilon(x) = \{j: |\phi_j(x)| \leq \varepsilon \text{ and } j = 1, \dots, l\}$ the set of ε -active equality constraints,

$$\mathbf{J}_A^\varepsilon(x) = \mathbf{J}_{A_1}^\varepsilon(x) \cup \mathbf{J}_{A_2}^\varepsilon(x).$$

The quantity ε is a parameter of the method that is set positive initially and is (possibly) revised downward at certain times during the course of the algorithm. Similarly μ is set initially to 1.0, and it may be revised downward.

We started with x^0 and

$$B^0 = J(x^0)^T J(x^0).$$

We divided the technique for determining B^k and x^k into a global, a dropping, an intermediate, and a local mode, each characterized by the use of different quadratic subproblems and/or a different selection of step directions.

The form of the quadratic subproblem is always given by

$$(5.2.1) \quad \begin{aligned} &\text{minimize}_h \nabla \Psi_\varepsilon(x^k)^T h + \frac{1}{2} h^T B^k h, \\ &\text{subject to } \nabla \phi_j^T(x^k) h = 0, \quad j \in \mathbf{J}_A^\varepsilon(x^k), \end{aligned}$$

but B^k approximates either $\nabla^2 \Psi_\varepsilon(x^k)$ or else $\nabla^2 L_\varepsilon(x^k, \lambda^k)$, depending upon which mode is currently in force. The modes are determined by letting

$$A^k \equiv [\dots, \nabla \phi_j(x^k), \dots] \quad \text{for } j \in \mathbf{J}_A^\varepsilon(x^k),$$

by selecting vectors z_l to form an orthonormal basis for the null space of A^{k^T} , by constructing the matrix Z^k with columns z_l , and then by inspecting the norm

$$(5.2.2) \quad \|Z^{k^T} \nabla \Psi_\varepsilon(x^k)\|_2.$$

Our code was based upon the version of the exact penalty algorithm given in [3], and our interpretation of that version is distinguished from the publication version in one respect: by the presence of the intermediate mode, which serves to provide a more cautious transition from the global algorithm of [5] and its asymptotic variant, discussed in [6].

Global mode. B^k is taken to approximate $\nabla^2 \Psi_\varepsilon(x^k)$ for this mode—its updating will be discussed later. Whenever (5.2.2) is greater than or equal to a tolerance τ_1 , then the global mode is used, and a direction h^k is found by solving (5.2.1). A new point

$$x^{k+1} = x^k + \alpha^k h^k$$

is found by using the line search of [24] with $\eta = 0.9$ on

$$\psi_\varepsilon(\alpha) = \Psi_\varepsilon(x^k + \alpha^k h^k).$$

Dropping mode. If (5.2.2) is less than τ_1 , then

Lagrange multiplier estimates λ^k are obtained by solving

$$(5.2.3) \quad \text{minimize}_\lambda \|A^k \lambda - \nabla \Psi_\varepsilon(x^k)\|_2,$$

and a test is made to see whether any $\lambda_{j_k}^k$ exists satisfying

$$(5.2.4) \quad \begin{aligned} |\lambda_{j_k}^k| &\notin [0, +1], & j_k \in \mathbf{J}_{A_1}^e(x^k), & \text{ or} \\ |\lambda_{j_k}^k| &\notin [-1, +1], & j_k \in \mathbf{J}_{A_2}^e(x^k). \end{aligned}$$

In this event, h^k is found by solving the underdetermined linear equations

$$(5.2.5) \quad \begin{aligned} \nabla \phi_{j_k}^T h &= \pm 1, \\ \nabla \phi_j^T h &= 0, & j \in \mathbf{J}_A^k(x^k) \setminus \{j_k\}, \end{aligned}$$

the sign being chosen to provide descent for Ψ_e . A tolerance δ is used to accept or reject h^k as providing sufficient local descent:

$$[\nabla \Psi_e(x^k) + \theta_k \nabla \phi_{j_k}(x^k)]^T h^k < \delta,$$

where $\theta_k \in \{-1, +1, 0\}$ depending upon the membership of j_k in $\mathbf{J}_E$ or $\mathbf{J}_I$ and upon the sign chosen in (5.2.5).

If no λ satisfies (5.2.4), then the intermediate mode of the method is initiated. Otherwise a new point

$$x^{k+1} = x^k + \alpha^k h^k$$

is found as using the line search. If h^k does not pass the test of sufficient decrease, however, then ε , τ_1 , and another tolerance τ_2 (used below) are reduced, the index sets $\mathbf{J}$ are reassessed, and the global mode is put in force.

Intermediate mode. In the intermediate mode B^k is still taken to approximate

$$\nabla^2 \Psi_e(x^k),$$

and h^k is determined by (5.2.1). (5.2.3) is solved at the beginning of each iteration of the mode to provide values of λ_j^k , and the conditions (5.2.4) are checked. To remain in the intermediate mode, these conditions must fail to hold. (If any $\lambda_{j_k}^k$ can be found to satisfy (5.2.4), then the dropping mode is put into force.) Assuming that the intermediate mode is permitted to continue, h^k is determined from (5.2.1), and a line search is made on $\psi_e(\alpha) = \Psi_e(x^k + \alpha h^k)$. This yields the point $x^k + \alpha^k h^k$. The vector

$$\Phi(x^k + \alpha^k h^k) = [\cdot \cdot \cdot, \phi_j(x^k + \alpha^k h^k), \cdot \cdot \cdot]^T, \quad j \in \mathbf{J}_A^e(x^k)$$

is formed; a direction v^k is found by solving

$$(5.2.6) \quad A^{kT} v = -\Phi(x^k + \alpha^k h^k),$$

and the point

$$x^k + \alpha^k h^k + v^k$$

is considered. If a tolerance test (see [5], [6]) indicates that

$$\Psi_e(x^k + \alpha^k h^k + v^k) \text{ is sufficiently smaller than } \Psi_e(x^k),$$

then

$$x^{k+1} = x^k + \alpha^k h^k + v^k;$$

otherwise

$$x^{k+1} = x^k + \alpha^k h^k;$$

$$\varepsilon, \tau_1, \text{ and } \tau_2$$

are reduced, and the global mode is put in force.

Local mode. If the norm (5.2.2) is less than a tolerance τ_2 ($\tau_2 < \tau_1$), and if $x^k + \alpha^k h^k + v^k$ has been accepted as x^{k+1} on the intermediate-mode iteration step last taken, then the local mode will be in force. In this mode B^k is maintained as a quasi-Newton approximation to

$$\nabla^2 L_\epsilon(x^k, \lambda^k),$$

and a direction h^k is determined from (5.2.1). The quantity $\alpha = \alpha^k$ is set to 1 without a line search, and the vector v^k is determined from (5.2.6) with this version of B^k . The point

$$x^{k+1} = x^k + h^k + v^k$$

is accepted, provided a tolerance test indicates that $\Psi_\epsilon(x^k + h^k + v^k)$ shows sufficient decrease over $\Psi_\epsilon(x^k)$. If this point is not accepted, then the intermediate mode is put into force after

$$\epsilon, \tau_1, \text{ and } \tau_2$$

have been reduced.

Updating. The quasi-Newton updating for

$$B^k = \mu J(x^k)^T J(x^k) + S^k$$

must take account of the fact that

$$S^k \approx \nabla^2 \Psi_\epsilon(x^k) - \mu J(x^k)^T J(x^k)$$

in the global and intermediate mode, and that

$$S^k \approx \nabla^2 L_\epsilon(x^k, \lambda^k) - \mu J(x^k)^T J(x^k)$$

in the local mode. This is attempted by setting

$$\begin{aligned} y^{k+1} &\leftarrow \mu [J(x^{k+1}) - J(x^k)]^T F(x^{k+1}) \\ &\quad - \sum_{j \in \mathbf{J}_{\Psi_1}^{\epsilon}(x^{k+1})} [\nabla \phi_j(x^{k+1}) - \nabla \phi_j(x^k)] \\ (5.2.7) \quad &+ \sum_{j \in \mathbf{J}_{\Psi_2}^{\epsilon}(x^{k+1})} \text{sgn}(\phi_j(x^{k+1})) [\nabla \phi_j(x^{k+1}) - \nabla \phi_j(x^k)] \\ &\text{+if \{local mode\} then } \sum_{j \in \mathbf{J}_{\lambda_1}^{\epsilon}(x^{k+1})} \lambda_j^{k+1} [\nabla \phi_j(x^{k+1}) - \nabla \phi_j(x^k)] \\ &\text{else 0,} \end{aligned}$$

and

$$s^{k+1} \leftarrow x^{k+1} - x^k.$$

The vectors y^{k+1} and s^{k+1} are used in the BFGS formula to update S^k to S^{k+1} (switching to DFP if division by zero is to be avoided).

In any event, the solution of (5.2.1), for B^k obtained in this fashion, is accomplished in all modes (global, intermediate, and local) via the LDL^T decomposition described in (5.1.1). It is important to note in this context that we were maintaining and updating the full B matrix, and then projecting it. This is *not* what is done in [5], [6]. There the authors advocate the use of an approximation *only* to the matrix $Z^T B Z$. The structure of our partial Hessian updates above, we felt at the time, mitigated against dealing easily with an approximation to the projected Hessian alone.

The decision was made to investigate the sensitivity of our interpretation of this method to indefinite Hessians. The next section describes a number of problems which were generated for least-squares problems with such Hessians.

6. Test results. The nonlinear programs to be considered here have the form of nonlinear least-squares problems:

$$\underset{x}{\text{minimize}} \frac{1}{2}F(x)^T F(x) + \Omega_0(\frac{1}{2}(x - x^*)^T D(x - x^*)) + \gamma_0((x - x^*)^T h_0)$$

subject to

$$\phi_j(x) = q_j(x) + h_j^T(x - x^*) + \alpha_j = 0, \quad j = 1, \dots, l,$$

$$\phi_j(x) = q_j(x) + h_j^T(x - x^*) + \alpha_j \geq 0, \quad j = l+1, \dots, l+m$$

where

$$F(x) = [f_1(x), \dots, f_p(x)]^T$$

with

$$f_i(x) = \sum_{k=1}^n (x_k^2 x_i) - i, \quad i = 1, \dots, p,$$

where

$$q_j(x) = \sum_{k=1}^n (x_k^2 x_j) - j, \quad j = 1, \dots, l+m,$$

and where

$$\Omega_0(t) = \gamma_0(t) - \frac{1}{2}(t+1)^2 \quad \text{for } t \in \mathbf{R}.$$

Note that this is consistent with the choice

$$\alpha_0 \equiv 0,$$

$$\gamma_j \equiv \text{identity} \quad \text{for } j > 0,$$

$$\delta_j \equiv 1,$$

$$\Omega_j \equiv 0 \quad \text{for } j > 0.$$

We have arranged the form of Ω_0 and γ_0 so that the objective function will be a sum of squares.

Test generator. A simple test-generating system (available from the authors) was written for the above. In all cases it assigned values to the optimizer and its associated Lagrange multipliers by uniformly distributed pseudo-random numbers. The components of x^* and the Lagrange multipliers λ^* of the active equality constraints were chosen between -1.0 and 1.0 , while the Lagrange multipliers of the active inequality constraints were chosen between 0.0 and 1.0 .

The Lagrangian Hessian was set to be

$$\nabla^2 L = \begin{bmatrix} \sigma U_1^T U_1 & 0 \\ 0 & U_2^T U_2 \end{bmatrix},$$

where U_1 was a $(\nu + l) \times (\nu + l)$ upper triangular matrix.

Except for $U_1[1, 1]$, which was set to 2, all elements of U_1 were generated by using a random number generator and were in the range -1.0 to 1.0 . U_2 was also an upper triangular matrix with elements all generated between -1.0 and 1.0 .

The scalar σ was under user control, and it dictated whether the generated problem would be definite ($\sigma > 0$) or indefinite ($\sigma \leq 0$). A sequence of problems could be generated in which all quantities (x^* values, Lagrange multipliers, U_1 , etc.) were exactly the same, and only $\nabla^2 L$ differed by having a different value of σ .

A feature for attempting degenerate optimizers was built in. Normally the gradient $\phi_j(x^*)$ was set to the j th column of the identity. A user-supplied option permitted the last several $\nabla \phi$'s in sequence to be constructed as random linear combinations of the initial $\nabla \phi$'s.

The constants α_j were set so that $\phi_j(x^*) = 0$ for the active constraints, and $\phi_j(x^*) = 1$ for the inactive constraints.

Problems. Five problems will be reported upon, the first three with $\sigma = 1, -1$, and -10 , respectively, and the last two with $\sigma = 1$ and -1 . For the first three values of σ , each problem was constructed to be nondegenerate at the optimizer, and for the remaining two values of σ each problem was made to be degenerate. To obtain degenerate problems, the gradient of the fourth active constraint was chosen to be linearly dependent on the gradients of the first three active constraints. The point x^* was verified computationally to be a local optimizer for all problems.

For the problems reported upon below, we had

$n = 5$ —number of variables

$p = 5$ —number of components in $F(x)$

$l = 2$ —number of equality constraints

$m = 3$ —number of inequality constraints

$\nu = 2$ —number of active inequality constraints

$$\text{Optimizer } x^* = \begin{pmatrix} 0.30467 \\ 0.62076 \\ -0.80999 \\ 0.41441 \\ 0.95425 \end{pmatrix}.$$

Active constraints set = $\{1, 2, 3, 4\}$.

$$\text{Lagrange multipliers } \lambda^* = \begin{pmatrix} 0.12629 \\ 0.59378 \\ 0.31597 \\ 0.58508 \end{pmatrix}.$$

Implementations. All codes, those for the test generation as well as those for the optimization methods, were written in ANSI FORTRAN, as verified by the PFORT verifier [28], and were run on the Honeywell 66/60 of the Mathematics Faculty Computing Facility at the University of Waterloo. The basic linear algebra computations (dot products, etc.) were carried out using the Basic Linear Algebra Subroutines of [19], and the more advanced linear algebra (e.g. forming QR factorizations and finding the LDL^T factorization of a—possibly nonnegative-definite—symmetric matrix) were carried out by using or modifying programs from the NPL optimization library [14]. The quadratic programming code needed for the method of [26] was also obtained from this library, which was made available through the courtesy of P. E. Gill and W. Murray. The line search routine that was used in all codes comes from reference [24] and was kindly provided by Michael Overton. All random numbers were produced by [33] as modified for double precision.

Employment. We illustrate the use of these test problems in two typical ways, firstly in the “shotgun” mode of software testing, in which the problems are given to the codes and the results are summarized in tabular or statistical format, secondly in a “scalpel” mode, in which selected problems are followed, step-by-step, through an algorithm to see what insights can be gained. For the former mode, the computational results are given in tables that follow. Unless otherwise indicated, all tests were terminated if more than 50 iterations were taken, in the interests of saving our computing budget. For the latter mode, we include summary remarks after each table indicating a few of the interesting features of the execution. Again, we remind the reader that our remarks reflect studies of the methods of [5], [6], [26] as subjected to our interpretation, for a problem of special structure, applying quasi-Newton updates of our choice, and suffering under our blind spots in rendering these methods into computer code. It is the purpose of the test examples to indicate where we might have fallen down on our task of implementing the methods.

Table headings:

EP—exact penalty method

SQP—successive quadratic programming method with watchdog

NI—number of iterations

SP—starting point:

starting point 1— x with all component equal to 1.0

starting point 2— x with all component equal to 10.0

starting point 3— x with all component equal to 100.0.

FE—number of objective function evaluations

(each evaluation of the objective function was counted as $p + 2$)

CE—number of constraint evaluations

GFE—number of gradient evaluations in the objective function

(each Jacobian evaluation was counted as $p + 2$)

GCE—number of gradient evaluations in the constraints

R—result:

C—Constructed optimizer found

A—Another point of termination

F—Failure, code aborted

M—Maximum iteration count reached

Tables:

TABLE 1
Nondegeneracy with positive-definite H ($\sigma = 1.0$).

SP	EP						SQP					
	NI	FE	CE	GFE	GCE	R	NI	FE	CE	GFE	GCE	R
1	22	245	175	210	169	C	19	147	105	105	103	C
2	35	490	350	435	375	C	10	98	70	70	65	F
3	38	497	355	420	448	C	47	434	310	310	298	C

Remarks:

(1) In SQP with starting point 1 the first 4 penalty parameters were determined by the Lagrange multipliers of the first quadratic subproblem, and the last penalty

parameter was determined by a multiplier of the second quadratic subproblem. The penalty parameters ranged from 51 to 350 and remained unchanged throughout iterations 3–19.

(2) In SQP with starting point 2, the penalty parameters were set between 16,750 and 89,016 by the first quadratic subproblem. At iteration 6, a quadratic-programming subproblem was encountered with Lagrange multipliers of the order of 10^9 . This forced the penalty parameters to become large as well. On the next two iterations, multipliers became 10^{23} and 10^{36} respectively. The line search became unable to find lower merit function values with the search directions produced, and the code was aborted.

(3) In SQP, with starting point 3, the penalty parameters had been set to numbers of magnitude 10^6 by iteration 3. These values held throughout the subsequent iterations without change.

(4) It is a general observation that the first few quadratic programs encountered by SQP for all of our starting points and for all of our generated problems were associated with large Lagrange multipliers. These, in turn, set the values of the penalty parameters quite large, and subsequent quadratic programs, with their much more reasonably sized multipliers, had no influence on the magnitude of the parameters.

(5) Each of the three executions of the EP method was characterized by an initial number of steps in which all constraints were deemed active, causing (very large) Lagrange multipliers to be computed. These generated “dropping directions”, (5.2.5). There followed an intermediate number of global-mode iterations in which steps were obtained from (5.2.1), during which $\|Z^T \nabla \Psi_\epsilon\|$ decreased almost monotonically. These steps gathered the activities that represented the optimal constraint manifold, and ended when the intermediate mode was entered. After several intermediate mode iterations, the local mode was begun. Although the theory of [6] only guarantees 2-step superlinear convergence, convergence to the constructed optimizer appeared to be 1-step superlinear. For starting point 2, for example, the breakdown was: 7 iterations of dropping steps, 16 of global steps, 3 intermediate steps, and 9 local steps. The components of x^k agreed with those at termination in the final 5 steps respectively to 3, 4, 7, 12, and >15 digits.

TABLE 2
Nondegeneracy with mildly indefinite $H(\sigma = -1.0)$.

SP	EP						SQP					
	NI	FE	CE	GFE	GCE	R	NI	FE	CE	GFE	GCE	R
1	23	252	180	205	159	C	22	182	130	130	126	C
2	32	455	325	385	334	C	10	98	70	70	65	F
3	44	518	370	395	347	A	50	672	480	480	464	M

Remarks:

- (1) The SQP method failed in the same manner as for starting point 2 in the previous problem—quadratic programs were encountered with large Lagrange multipliers.
- (2) For starting point 3, the EP method terminated on a point (not the constructed optimizer) that satisfied the first-order necessary conditions. The code does not check second-order conditions.

TABLE 3
Nondegeneracy with strongly indefinite $H(\sigma = -10.0)$.

SP	EP						SQP					
	NI	FE	CE	GFE	GCE	R	NI	FE	CE	GFE	GCE	R
1	50	1,015	830	675	634	A	19	140	100	100	99	C
2	50	868	625	715	646	M	10	98	70	70	65	F
3	50	798	590	645	577	M	33	238	170	170	169	C

Remarks:

- (1) The EP code arrived at a stationary point (a first-order point that was infeasible for the given constraints) on iteration 30. Progress to this stage involved an initial mixture of global and dropping steps, an attempted intermediate step followed by several successful intermediate steps, and a final number of local steps. The stationary point had constraints 2 and 4 active (1, 2, 3, and 4 are active at the constructed optimizer), and was infeasible in constraint 1. The penalty parameter μ was reduced to an eighth of its value, and the remaining 20 iterations brought convergence to a point that satisfied the first-order conditions. This point was $[0.299, 0.610, -0.740, 0.407, 0.87]$, which is close to the constructed optimizer, but it has only constraints 1, 2, and 4 active (values $\approx 10^{-20}$). Constraint 3, which is zero at the constructed optimizer, has the value 0.0401 at this point. There is a good chance that the projected Hessian of the Lagrangian—by the nature of the Hessian’s construction, by the proximity of this point to the constructed optimizer, and by the fact that the gradient of constraint 3 is not included in the projection—is indefinite. Our code, which maintains a full approximation to $\nabla^2 L$ rather than an approximation to $Z^T \nabla^2 L Z$ appeared sensitive to indefiniteness in the Lagrangian Hessian. This was further emphasized by poor performance on the problem of Table 5.
- (2) The EP method behaved much the same way on starting points 2 and 3, but lagged enough behind its progress on starting point 1 that it was cut off at iteration 50 just on or after a reduction in μ .
- (3) SQP failure was as before.

TABLE 4
Degeneracy with positive-definite $H(\sigma = 1.0)$.

SP	EP						SQP					
	NI	FE	CE	GFE	GCE	R	NI	FE	CE	GFE	GCE	R
1	33	392	280	285	237	C	50	658	470	470	445	M
2	50	791	656	615	555	M	50	728	520	520	516	M
3	49	588	420	425	455	C	22	1,519	1,085	1,085	1,080	F

Remarks:

- (1) With starting points 1 and 3, EP converged to the optimizer. The Lagrange multipliers at the termination point were in agreement with the first-order conditions,

but they were different in value from those assigned due to the linear dependence of the constraint gradients.

- (2) For starting point 2, EP attained a stationary point, reduced μ , and was making normal progress when iteration 50 was reached.
- (3) The failures of SQP were as above.

TABLE 5
Degeneracy with mildly indefinite $H(\sigma = -1.0)$.

SP	EP						SQP					
	NI	FE	CE	GFE	GCE	R	NI	FE	CE	GFE	GCE	R
1	50	623	445	485	391	M	50	1,379	985	985	926	M
2	50	1,645	1,195	1,245	1,137	M	49	1,050	750	750	746	F
3	50	623	445	565	456	M	29	315	225	225	225	C

Remarks:

- (1) For starting point 1, the behaviour of the EP, which was noted above, was to be seen. A stationary point was reached, μ was reduced, and normal progress appeared to continue as iteration 50 was reached. To verify the impression that progress was normal, a second run was made with 100 iterations allowed. The method terminated at the constructed optimum after 95 iterations.
- (2) For starting point 2, the EP code reached a state at which only a minute amount of change in x was made at every iteration. The code had, to all intents, stalled, and it was aborted.
- (3) For starting point 3, the EP code attained a stationary point around iteration 50. It was cut off before a reduction in μ could take place.
- (4) The SQP method again developed large penalty parameters and ceased making progress.

Summary. We make note of the fact that the problems were easy to generate, that the generation process was flexible enough to handle a special request on the form of the objective function (i.e. that it be a sum of squares), that problems could be generated automatically, and that useful information pointing to the areas that need further study was obtained.

Acknowledgments. The authors wish to thank A. R. Conn, who has given freely of his suggestions and encouragement. Thanks are also due to R. H. Byrd and A. L. Goldman, who undertook careful readings of the thesis from which much of this material is derived, and to W. Murray, P. E. Gill, and M. L. Overton for providing the use of certain of their software routines. Finally, the authors are grateful to J. T. Hon for generating the problems reported here, running the algorithms, and creating the tables of this section.

REFERENCES

[1] Å. BJÖRCK AND T. ELFVING (1973), *Algorithms for confluent Vandermonde systems*, Numer. Math., 21, pp. 130–137.
[2] A. CHARNES, W. M. RAIKE, J. D. STUTZ AND A. S. WALTERS (1974), *On generation of test problems for linear programming codes*, Comm. ACM, 17(10), pp. 583–586.

- [3] T. F. COLEMAN (1979), *A superlinear penalty function method to solve the nonlinear programming problem*, Ph.D. dissertation, Dept. Combinatorics and Optimization, Univ. Waterloo, Waterloo, Ontario, Canada.
- [4] A. R. COLEVILLE (1970), *A comparative study on nonlinear programming codes*, in Proc. the Princeton Symposium on Mathematical Programming, H. W. Kuhn, ed., Princeton Univ. Press, Princeton, NJ.
- [5] A. R. CONN AND T. F. COLEMAN (1982), *Nonlinear programming via an exact penalty function: global analysis*, Mathematical Programming, 24, North-Holland, Amsterdam, pp. 137–161.
- [6] ———, (1982), *Nonlinear programming via an exact penalty function: asymptotic analysis*, Mathematical Programming, 24, North-Holland, Amsterdam, pp. 123–136.
- [7] H. CROWDER, R. S. DEMBO AND J. M. MULVEY (1978), *Reporting computational experiments in mathematical programming*, Mathematical Programming, 15, North-Holland, Amsterdam, pp. 316–329.
- [8] ———, (1979), *On reporting computational experiments with mathematical software*, ACM Trans. Math. Software, 5(2), pp. 193–203.
- [9] R. S. DEMBO (1976), *A set of geometric programming test problems and their solutions*, Mathematical Programming, 10, North-Holland, Amsterdam, pp. 192–213.
- [10] J. E. DENNIS, JR. AND J. J. MORÉ (1977), *Quasi-Newton methods: motivation and theory*, SIAM Review, 19, pp. 46–89.
- [11] J. E. DENNIS, JR., D. M. GAY AND R. E. WELSCH (1981), *An adaptive nonlinear least-squares algorithm*, ACM Trans. Math. Software, 7, pp. 248–383.
- [12] A. V. FIACCO AND G. P. MCCORMICK (1968), *Nonlinear Programming: Sequential Unconstrained Minimization Techniques*, John Wiley, New York.
- [13] P. E. GILL AND W. MURRAY (1978), *Numerically stable methods for quadratic programming*, Mathematical Programming, 14, North-Holland, Amsterdam, pp. 349–372.
- [14] P. E. GILL, W. MURRAY AND S. PICKEN (1978), *The NPL Numerical Optimization Software Library*, Division of Numerical Analysis and Computer Science, National Physical Laboratory, Teddington, Middlesex, TW11 0LW, England.
- [15] S-P. HAN (1976), *Superlinearly convergent variable metric algorithms for general nonlinear programming problems*, Mathematical Programming, 11, North-Holland, Amsterdam, pp. 263–282.
- [16] ———, (1977), *A globally convergent method for nonlinear programming*, J. Optim. Theory Appl., 22(3), pp. 297–310.
- [17] D. M. HIMMELBLAU (1972), *Applied Nonlinear Programming*, McGraw-Hill, New York.
- [18] K. L. HOFFMAN AND D. R. SHIER (1980), *A test problem generator for discrete linear L1 approximation problems*, ACM Trans. Math. Software, 6, pp. 587–593.
- [19] C. L. LAWSON, R. J. HANSON, D. R. KINCAID AND F. T. KROGH (1979), *Basic linear algebra subprograms for FORTRAN usage*, ACM Trans. Math. Software, 5, pp. 308–325.
- [20] J. H. MAY AND R. L. SMITH (1980), *Random polytopes: their definition, generation and aggregate properties*, 80–6, Dept. Industrial and Operations Engineering, Univ. Michigan, Ann Arbor.
- [21] W. M. MICHAELS AND R. O'NEILL (1980), *A mathematical program generator MPGENR*, ACM Trans. Mathematical Software, 6, pp. 31–44.
- [22] J. J. MORÉ (1978), *Implementation and testing of optimization software*, IFIP WG 2.5, Working Conference on Performance Evaluation of Numerical Software, December, Baden, Austria.
- [23] J. J. MORÉ, B. S. GARBOW AND K. E. HILLSTROM (1981), *Algorithm 566: FORTRAN subroutines for testing unconstrained optimization software*, ACM Trans. Mathematical Software, 7, pp. 136–140.
- [24] W. MURRAY AND M. L. OVERTON (1978), *Steeplength algorithms for minimizing a class of non-differentiable functions*, STAN-CS-78-679, Computer Science Dept, Stanford Univ., Stanford, CA, November.
- [25] W. MURRAY AND M. H. WRIGHT (1978), *Projected Lagrangian methods based on the trajectories of penalty and barrier functions*, SOL 78-23, Operations Research Dept., Systems Optimization Laboratory, Stanford Univ., Stanford, CA.
- [26] M. J. D. POWELL, R. M. CHAMBERLAIN, C. LEMARECHAL AND H. C. PEDERSEN (1979), *The watchdog technique for forcing convergence in algorithms for constrained optimization*, Tenth International Symposium on Mathematical Programming, Montreal, Canada, August.
- [27] J. B. ROSEN AND S. SUZUKI (1965), *Construction of nonlinear programming test problems*, Comm. ACM, 8(2), p. 113.
- [28] B. G. RYDER (1974), *The PFORT verifier*, Software Practice and Experience, 4, pp. 359–377.
- [29] R. W. H. SARGENT (1982), *Recursive quadratic programming algorithms and their convergence properties*, Numerical Analysis, Proceedings, Cocoyoc, Mexico 1981, J. P. Hennert, ed., Lecture Notes Mathematics, 909, Springer-Verlag, Berlin.

- [30] K. SCHITTKOWSKI (1979), *Randomly generated nonlinear programming test problems*, 9th IFIP Conference on Optimization Techniques, Warszawa.
- [31] ———, (1980), *A numerical comparison of 13 nonlinear programming codes with randomly generated test problems*, in Numerical Optimization of Dynamic Systems, L. C. W. Dixon and G. P. Szego, eds., North-Holland, Amsterdam.
- [32] K. SCHITTKOWSKI AND W. HOCK (1981), *Test examples for nonlinear programming codes*, Lecture Notes in Economic and Mathematical Systems 187, Springer-Verlag, Berlin.
- [33] L. SCHRAGE (1979), *A more portable FORTRAN random number generator*, ACM Trans. Math. Software, 5, pp. 132–138.
- [34] G. W. STEWART, (1981), *Constrained definite Hessians tend to be well conditioned*, Mathematical Programming, 21, North-Holland, Amsterdam, pp. 235–238.
- [35] J. TELGEN AND W. B. VAN DAM (1979), *Randomly generated polytopes for testing mathematical programming algorithms*, 7929/0, Economics Institut, Erasmus University, Holland.